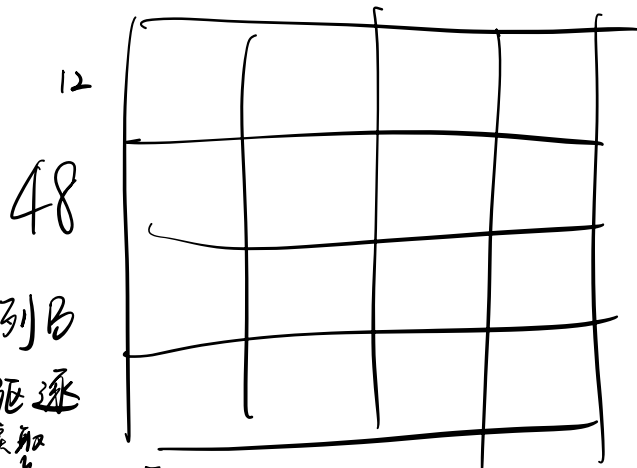
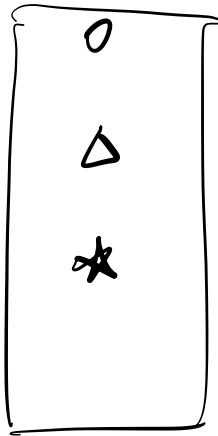
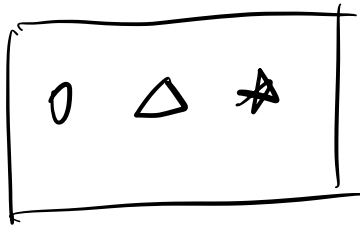


trans.c

转置



先读一行A再写一列B
防止B把A数据驱逐
后再读取
但无法避免下一行A将B驱逐

$$48 \times 4 = 192 \text{ 字节}$$

$$\frac{192}{48} = 4 \text{ 副本/步长}$$

$$\frac{48}{4} = 12 \Rightarrow \text{列} = 12$$

$$\frac{48}{4} = 12 \Rightarrow \text{行} = 12$$



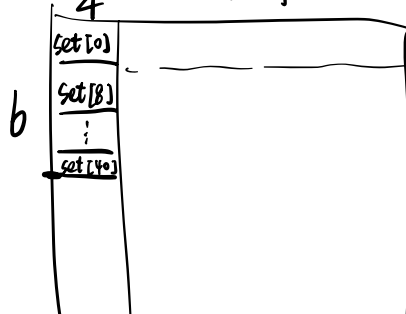
12x12

48

$$\frac{96 \times 4}{48} = 8$$

$$\frac{48}{8} = 6 \Rightarrow \text{列} = 6$$

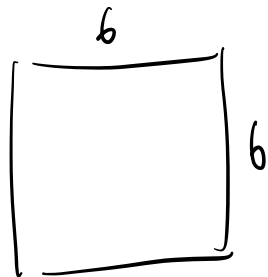
$$\frac{48}{4} = 12 \Rightarrow \text{行} = 12$$



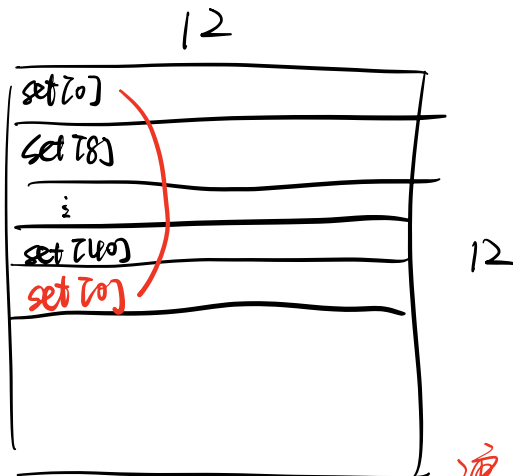
12x12

96

冲突
⇒



未能充分利用 1 个 block
只使用 6 个 reg



浪费的

$2 \times 2 = 4$ 倍 WB 部分

每 6 行就要冲突

相比理想中 12×12 浪费的

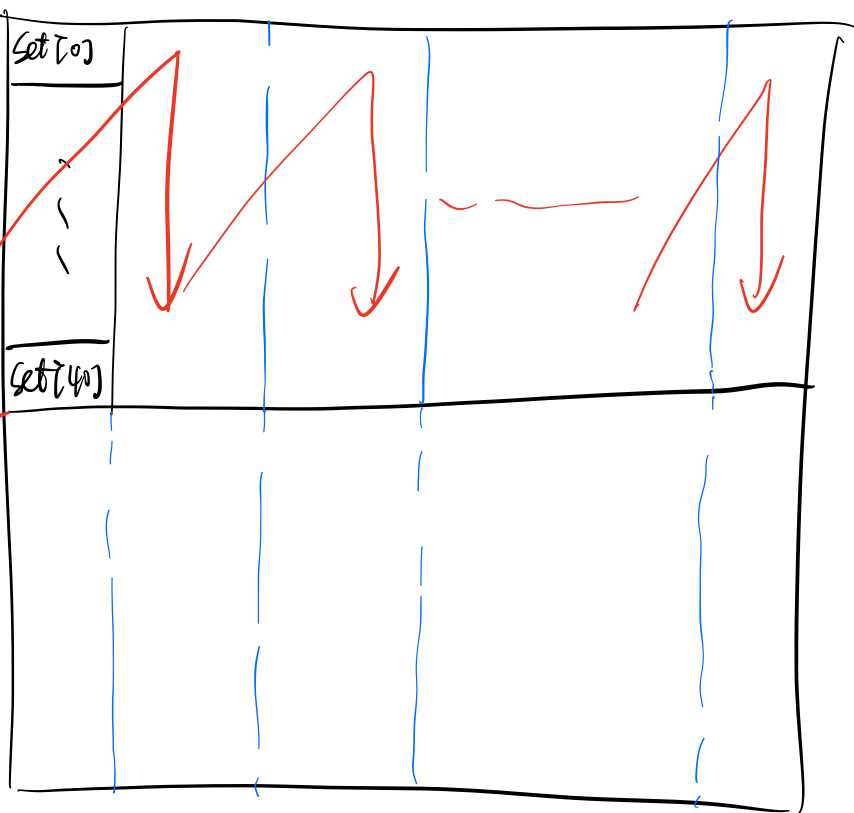
$2 \times 2 = 4$ 倍 LA 部分
行列

策略



① 先写前 6 个

② 再写后 6 个



但是：② 时又不得不重新读一遍 A (被 B 驱逐)

于是考虑：有没有办法，只写前 6 行，但写入完整 A

这样避免再读一遍？

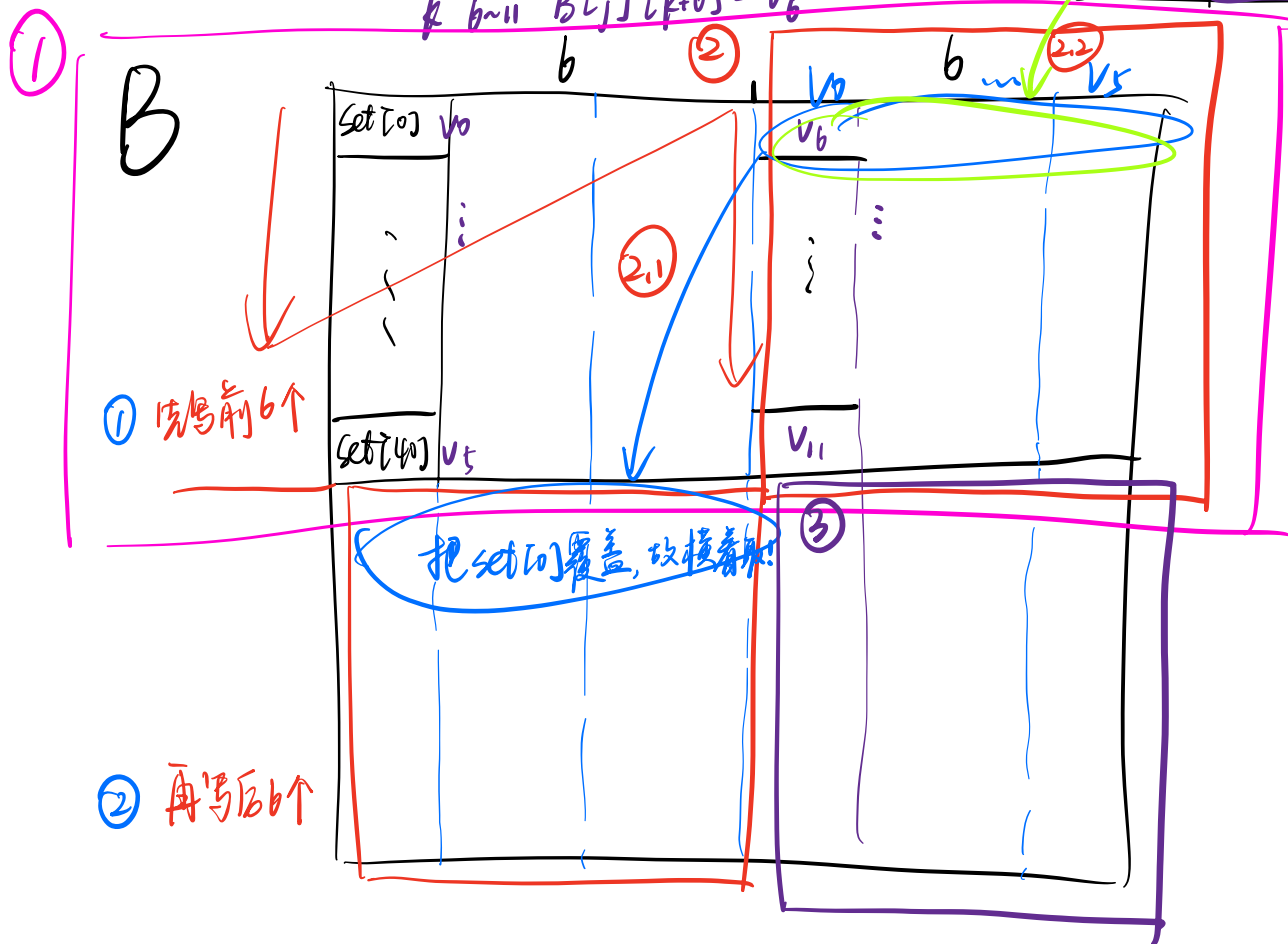
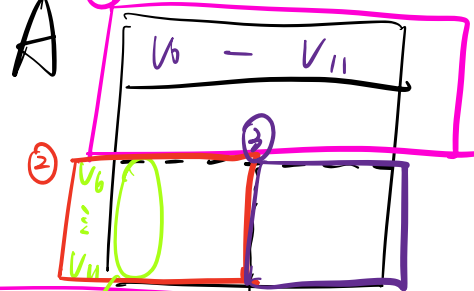
①

最终策略:

$$V_0 = A[k][j]$$

$$k \sim 5 \quad B[j][k] = V_0$$

$$k \sim 11 \quad B[j][k+b] = V_0$$



优点是: 实现 Cache 无缝衔接

① $A_{上} + B_{上}$ ← 覆盖

⇒ ② $A_{下} + B_{下}$

⇒ ③ $A_{下} + B_{下}$

估算: 四块完整 cache
(A上下, B上下) ☆

(实际略大于估算, 因为 A下可能驱逐 B上/下
但由于牺牲部分 B_上, 而 B_上 已处理, 故实际还不错)

相抵法二:

四块完整 cache
+ 额外 1 次 A 读取

Honor Part

1° 用线代知识优化

2° 分块

$$S=32, B=32$$

↓

$$A, B \text{ 列} = 8$$

$$A [32] [32]$$

$$\frac{32 \times 4}{32} = 4$$

$$\frac{32}{4} = 8$$

$$B \text{ 行} = 8$$

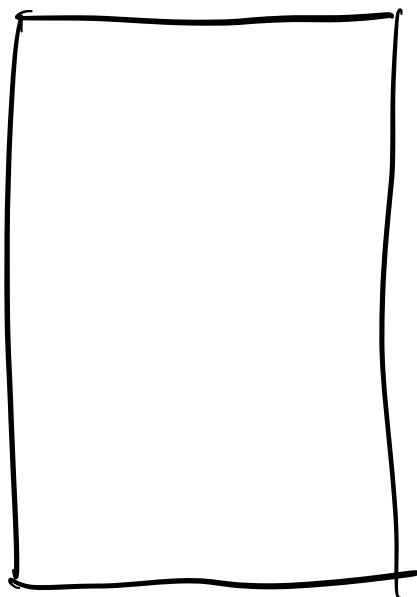
对应代码:

$$A \text{ 分块 } k^i \times 8^k$$

$$B \text{ 分块 } 8^k \times 8^j$$

$$\Rightarrow C \text{ 分块 } k \times 8$$

A



B

set [0]
set [4]
set [28]

希望尽量减少冲突，

而B每隔4 set，A为连续k set
可设k为4，保证不会冲突2组B

经测试，
证明了

效果优于 $k=2/4/16$
(代码中k=8)